FPGA Development for Radar, Radio-Astronomy and Communications

THE RADAR MASTERS COURSE

Dept. Electrical Engineering, University of Cape Town
Private Bag, Rondebosch, 7701, South Africa
http://www.rrsg.uct.ac.za

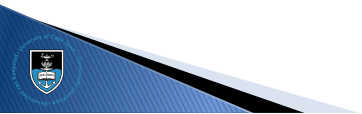
Presented by Dr John-Philip Taylor

Convened by Dr Stephen Paine

Day 5 – 7 August 2026

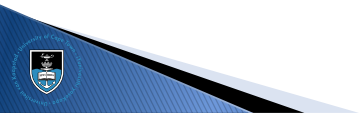
Outline

Outline



Typical Workflow

1. Design the IIR filter in Matlab / Octave / Python
   (pay careful attention to potential rounding errors and the resolution required)
2. Implement the IIR filter on the sample clock
3. Verify by means of simulation
4. Integrate the IIR filter into the system



IIR Filter

$$H(s) = \frac{\omega_0^2}{s^2 + 2\zeta\omega_0 s + \omega_0^2}, \quad \omega_0 = 2\pi f_0, \quad \zeta = \frac{1}{\sqrt{2}}$$

$$s = (1 - z^{-1}) f_s, \quad f_s = 781.25 \text{ kHz}$$

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\omega_0^2}{a - bz^{-1} + cz^{-2}}, \quad \begin{cases} a = f_s^2 + 2\zeta\omega_0 f_s + \omega_0^2 \\ b = 2f_s^2 + 2\zeta\omega_0 f_s \\ c = f_s^2 \end{cases}$$

$$Y(z)\left(a - bz^{-1} + cz^{-2}\right) = X(z)\left(\omega_0^2\right)$$

$$y_n = \left(\frac{2^N}{a}\right)\left(\frac{\omega_0^2 x_n + b y_{n-1} - c y_{n-2}}{2^N}\right)$$

IIR Filter

$$H(s) = \frac{\omega_0^2}{s^2 + 2\zeta\omega_0 s + \omega_0^2}, \quad \omega_0 = 2\pi f_0, \quad \zeta = \frac{1}{\sqrt{2}}$$

$$s = (1 - z^{-1}) f_s, \quad f_s = 781.25 \text{ kHz}$$

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\omega_0^2}{a - bz^{-1} + cz^{-2}}, \quad \begin{cases} a = f_s^2 + 2\zeta\omega_0 f_s + \omega_0^2 \\ b = 2f_s^2 + 2\zeta\omega_0 f_s \\ c = f_s^2 \end{cases}$$

$$Y(z)\left(a - bz^{-1} + cz^{-2}\right) = X(z)\left(\omega_0^2\right)$$

$$y_n = \left(\frac{2^N}{a}\right)\left(\frac{\omega_0^2 x_n + b y_{n-1} - c y_{n-2}}{2^N}\right)$$

IIR Filter

$$H(s) = \frac{\omega_0^2}{s^2 + 2\zeta\omega_0 s + \omega_0^2}, \quad \omega_0 = 2\pi f_0, \quad \zeta = \frac{1}{\sqrt{2}}$$

$$s = (1 - z^{-1}) f_s, \quad f_s = 781.25 \text{ kHz}$$

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\omega_0^2}{a - bz^{-1} + cz^{-2}}, \quad \begin{cases} a = f_s^2 + 2\zeta\omega_0 f_s + \omega_0^2 \\ b = 2f_s^2 + 2\zeta\omega_0 f_s \\ c = f_s^2 \end{cases}$$

$$Y(z)\left(a - bz^{-1} + cz^{-2}\right) = X(z)\left(\omega_0^2\right)$$

$$y_n = \left(\frac{2^N}{a}\right)\left(\frac{\omega_0^2 x_n + b y_{n-1} - c y_{n-2}}{2^N}\right)$$

IIR Filter

$$H(s) = \frac{\omega_0^2}{s^2 + 2\zeta\omega_0 s + \omega_0^2}, \quad \omega_0 = 2\pi f_0, \quad \zeta = \frac{1}{\sqrt{2}}$$

$$s = (1 - z^{-1}) f_s, \quad f_s = 781.25 \text{ kHz}$$

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\omega_0^2}{a - bz^{-1} + cz^{-2}}, \quad \begin{cases} a = f_s^2 + 2\zeta\omega_0 f_s + \omega_0^2 \\ b = 2f_s^2 + 2\zeta\omega_0 f_s \\ c = f_s^2 \end{cases}$$

$$Y(z)\left(a - bz^{-1} + cz^{-2}\right) = X(z)\left(\omega_0^2\right)$$

$$y_n = \left(\frac{2^N}{a}\right)\left(\frac{\omega_0^2 x_n + b y_{n-1} - c y_{n-2}}{2^N}\right)$$

IIR Filter

$$H(s) = \frac{\omega_0^2}{s^2 + 2\zeta\omega_0 s + \omega_0^2}, \quad \omega_0 = 2\pi f_0, \quad \zeta = \frac{1}{\sqrt{2}}$$

$$s = (1 - z^{-1}) f_s, \quad f_s = 781.25 \text{ kHz}$$

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\omega_0^2}{a - bz^{-1} + cz^{-2}}, \quad \begin{cases} a = f_s^2 + 2\zeta\omega_0 f_s + \omega_0^2 \\ b = 2f_s^2 + 2\zeta\omega_0 f_s \\ c = f_s^2 \end{cases}$$

$$Y(z)\left(a - bz^{-1} + cz^{-2}\right) = X(z)\left(\omega_0^2\right)$$

$$y_n = \left(\frac{2^N}{a}\right)\left(\frac{\omega_0^2 x_n + by_{n-1} - cy_{n-2}}{2^N}\right)$$

IIR Filter

$$y_n = \frac{Ax_n + By_{n-1} - Cy_{n-2}}{2^N}, \quad \begin{cases} A = (2^N/a) \cdot \omega_0^2 \\ B = (2^N/a) \cdot b \\ C = (2^N/a) \cdot c \\ N = 32 \end{cases}$$

f_0	A	B	C
10	28	8 589 446 092	4 294 478 824
100	2 775	8 585 049 594	4 290 085 073
1 000	274 663	8 541 087 701	4 246 395 068
10 000	24 799 428	8 104 255 079	3 834 087 211
100 000	997 792 625	4 839 800 553	1 542 625 882

IIR Filter

- To get the IIR filter to work at low pass-band frequencies, you need LARGE internal word-lengths
- Be careful with overflows – perform a limit operation
- Be careful with the signedness of the operation
- To help you keep track of what bit represents what in fixed-point representations, use negative indices:

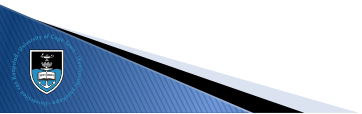
```
// Constants are in the range [0, 2), but
// Verilog still thinks they're unsigned integers
wire [0:-32]B = 33'd8_589_446_092;
// Internal registers are in the range [-1, 1)
reg  [0:-39]y_1;
// Products are in the range [-2, 2)
wire [1:-71]B_y_1;
```

IIR Filter

- To get the IIR filter to work at low pass-band frequencies, you need LARGE internal word-lengths
- Be careful with overflows – perform a limit operation
- Be careful with the signedness of the operation
- To help you keep track of what bit represents what in fixed-point representations, use negative indices:

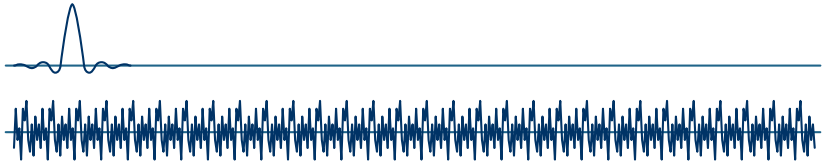
```
// Constants are in the range [0, 2), but
// Verilog still thinks they're unsigned integers
wire [0:-32]B = 33'd8_589_446_092;
// Internal registers are in the range [-1, 1)
reg  [0:-39]y_1;
// Products are in the range [-2, 2)
wire [1:-71]B_y_1;
```

Outline



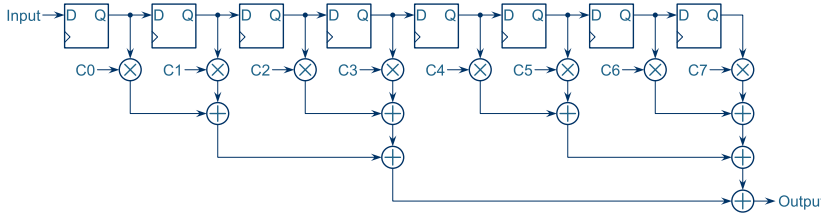
THE RADAR MASTERS COURSE

FIR Filter – Concept



- Convolve the impulse-response with the signal:
  1. Time-reverse the impulse-response
  2. Within the FIR filter window, multiply the impulse-response sample by the signal sample
  3. Sum the products and output the result
  4. Move the impulse-response by one sample
  5. Repeat from step 2

FIR Filter – Implementation

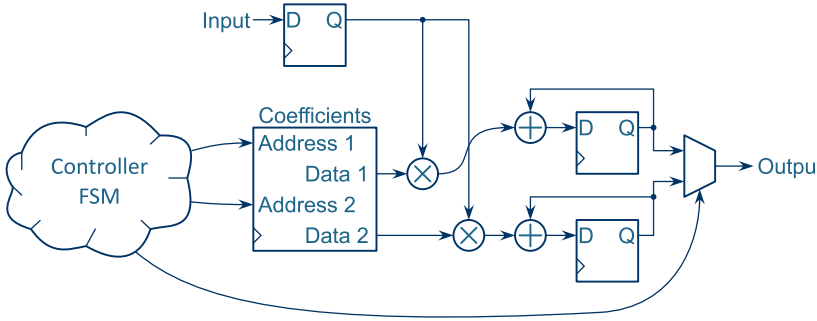


- Move the signal instead of the impulse-response
- To check the direction, inject an impulse (which should produce the impulse-response at the output)
- Pipeline the adder tree to increase the maximum clock frequency

Architecture Design

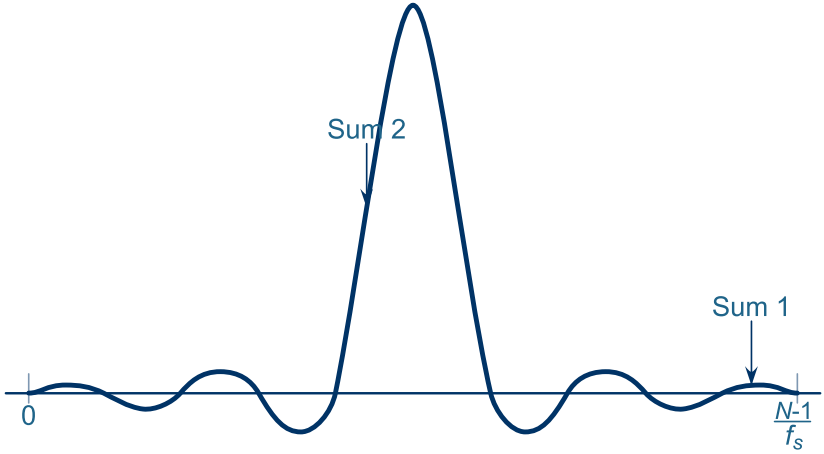
- Always take the design criteria into account when designing an architecture
- What happens when you add decimation
  (i.e. you don't need an output sample every clock cycle)?
- What if the FPGA clock is much faster than the sample rate?
- What about a combination of the above?
- Always consider the scenario: sample rate, clock speed, power requirements, throughput requirements, decimation (if any), available resources, etc...

Decimation

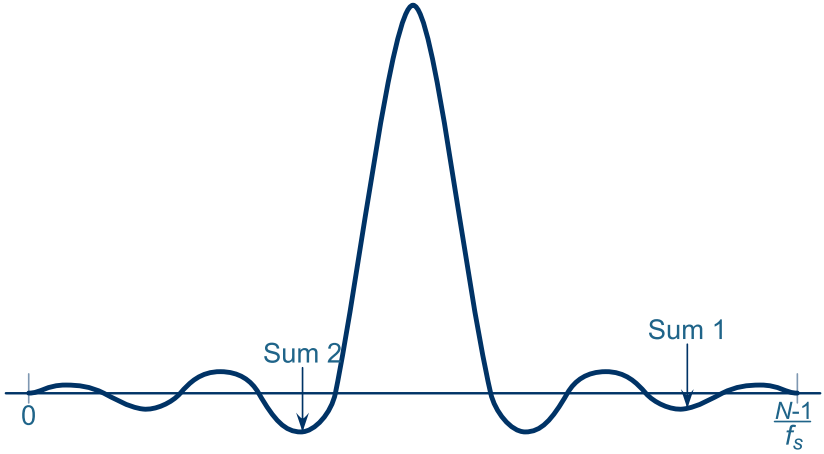


- This example decimates by $N/2$:
  a 512-point filter will decimate by 256
- The coefficient addresses are run $N/2$ out of phase

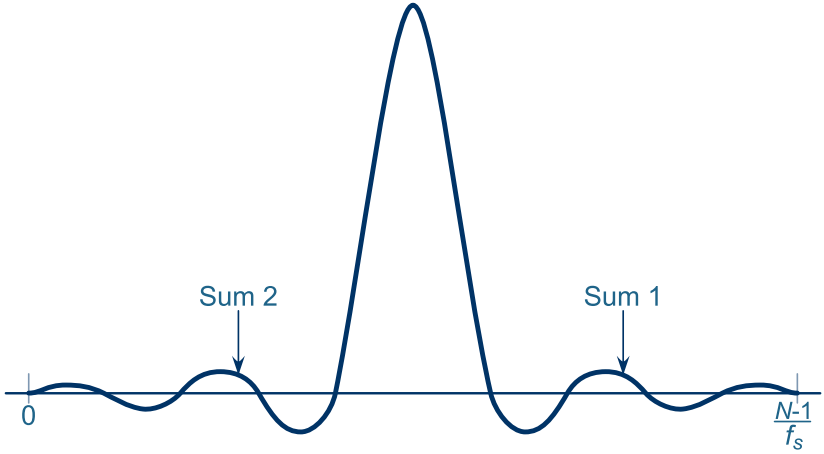
Decimation



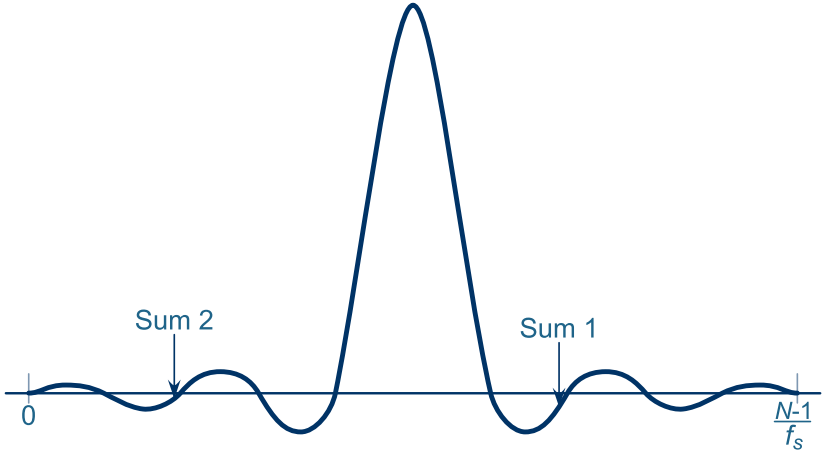
Decimation



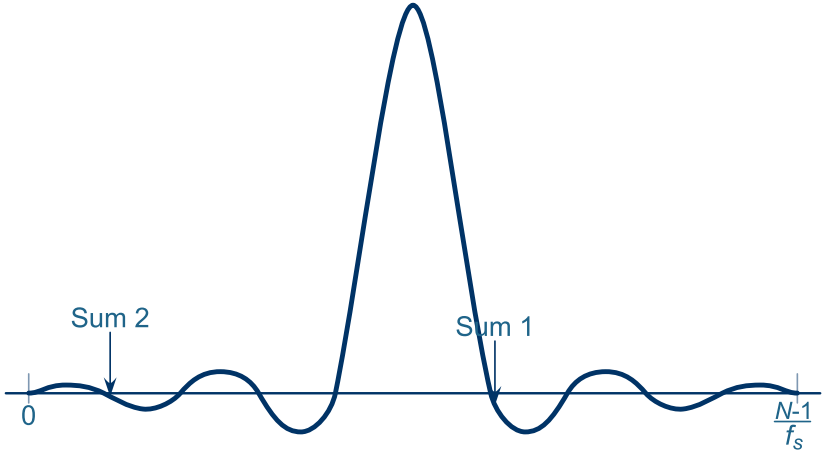
Decimation



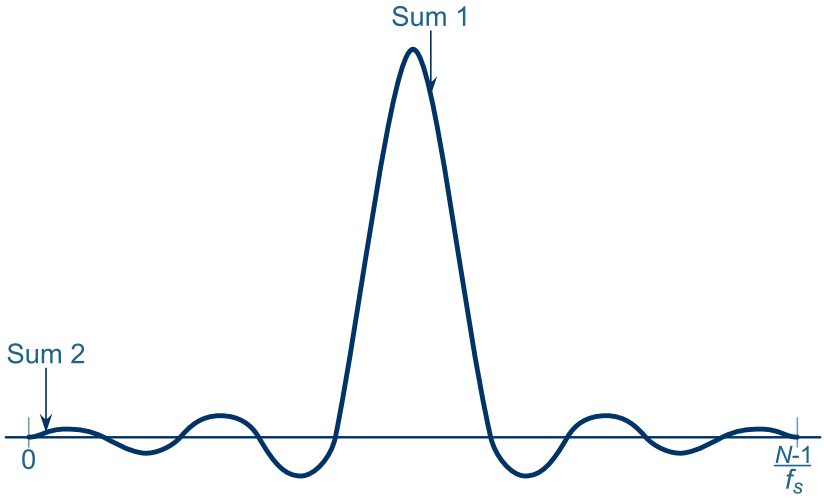
Decimation



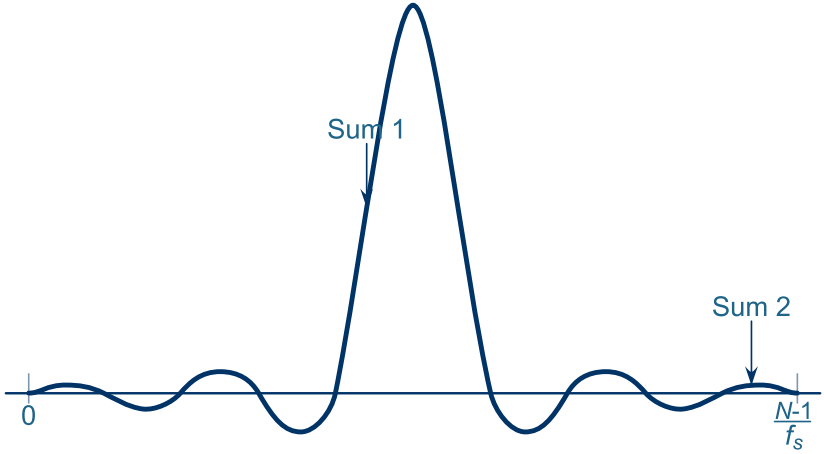
Decimation



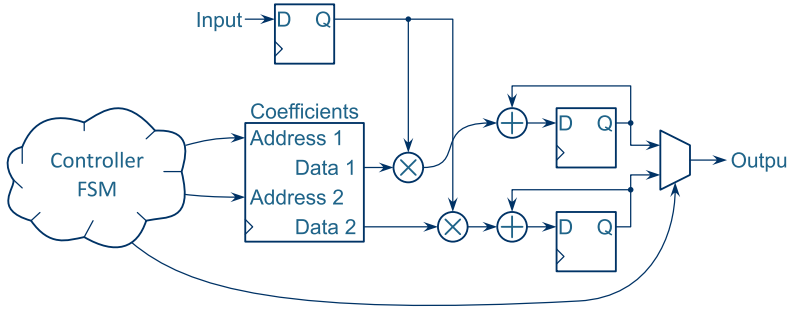
Decimation



Decimation

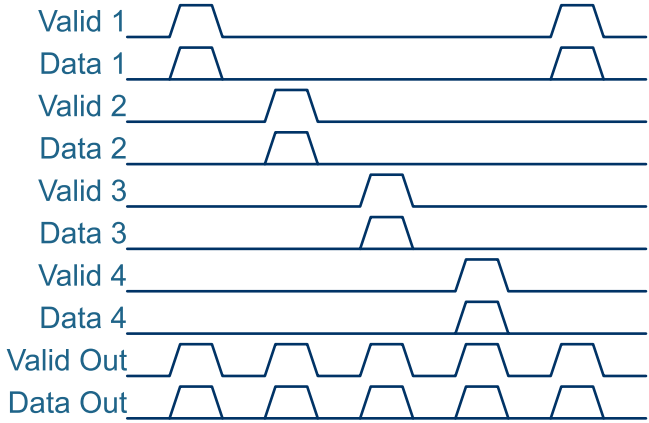


Faster System Clock

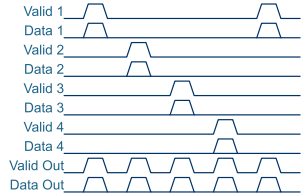


- If the sample clock is lower than the system clock, this same architecture can decimate by less
- Keep more than 2 sums (sometimes more efficient to keep these in BRAM as well)

Combining FIR Filter Units



FIR Filter



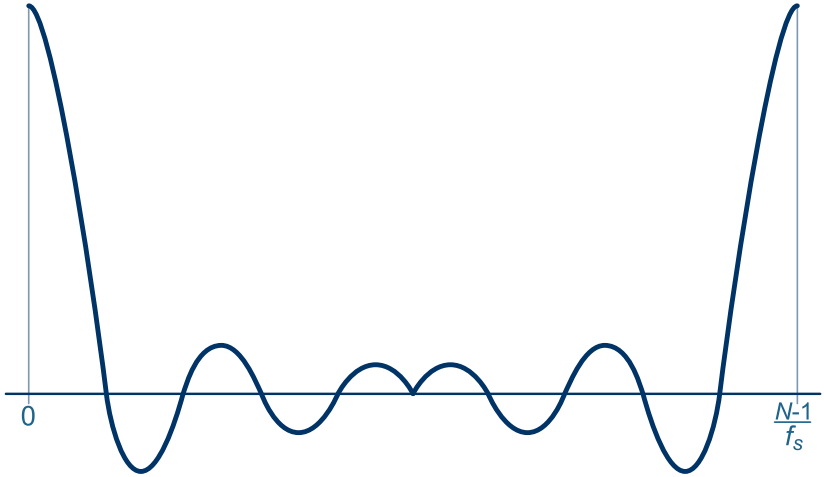
- Use multiple instances of a filter that decimates more than desired
- Reset the counters of the units out of phase
- Combine the outputs with a simple AND-OR circuit

Typical Workflow

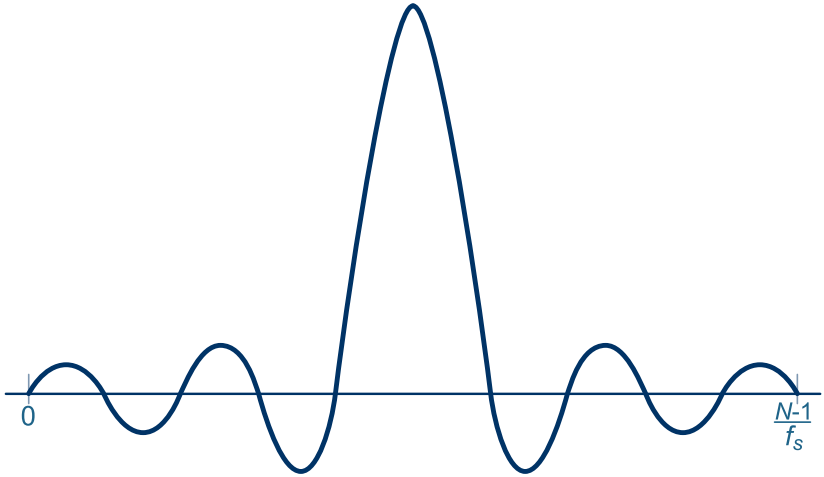
- Design the FIR filter in Matlab and test using integer values
- Check for overflows, rounding problems, etc.
- Design what bit-widths to use, given the native RAM and DSP elements of the FPGA in question
- Implement and integrate the FIR filter into the design, and test the system as a whole

FIR Filter – Desired Response

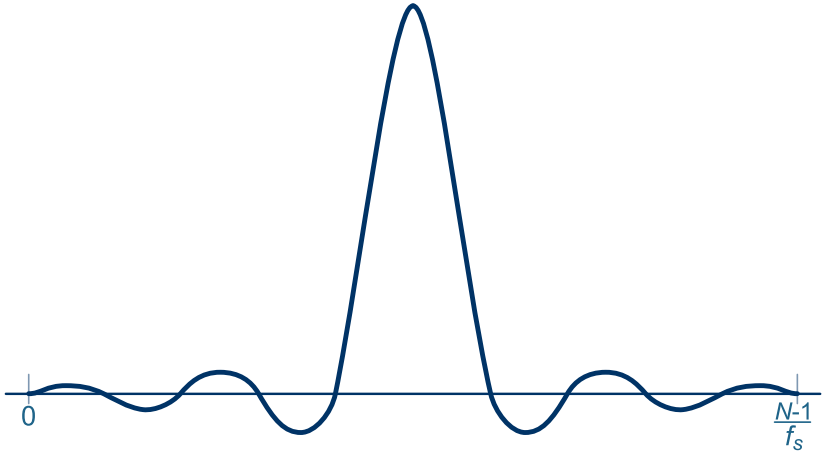
FIR Filter – IFFT (real part)



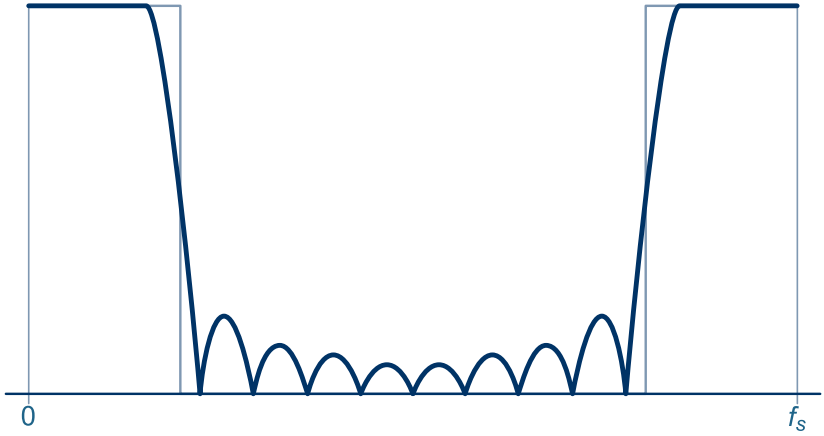
FIR Filter – Time-shifted



FIR Filter – Windowed



FIR Filter – Actual Response



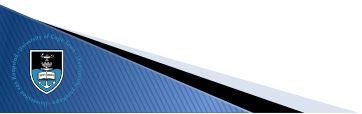
(Zero-pad to see the side-lobes)

FIR Filter Implementation

- Implement a N-point FIR filter that decimates by N (i.e. one sample output for every N samples input)
- Use an appropriate cut-off frequency and a Hann window $w(n) = \sin^2\left(\frac{\pi n}{N-1}\right)$
- Use Matlab / Octave / Python to generate the FIR filter constants and memory initialisation file
- Verify through simulation
- Verify on FPGA
- Combine eight filter units to drop the sub-sampling rate to N/8
- Verify on FPGA

Coffee Break...

Outline



Project Overview

- Everybody must do a different project: different microphone array, different number of microphones, different waveform, etc.
- You can propose a project: preferably in line with your current MSc research
- You need to design the DSP chain and choose appropriate system parameters: ADC sampling-rate, decimation (if any), architecture etc.
- Typically, you would build a sonar with angle-extraction, but you could also choose a communication or other type of project.
- Your system must be controllable from the PC (typically over the JTAG)

Project Overview

- Everybody must do a different project: different microphone array, different number of microphones, different waveform, etc.
- You can propose a project: preferably in line with your current MSc research
- You need to design the DSP chain and choose appropriate system parameters: ADC sampling-rate, decimation (if any), architecture etc.
- Typically, you would build a sonar with angle-extraction, but you could also choose a communication or other type of project.
- Your system must be controllable from the PC (typically over the JTAG)

Project Overview

- Everybody must do a different project: different microphone array, different number of microphones, different waveform, etc.
- You can propose a project: preferably in line with your current MSc research
- You need to design the DSP chain and choose appropriate system parameters: ADC sampling-rate, decimation (if any), architecture etc.
- Typically, you would build a sonar with angle-extraction, but you could also choose a communication or other type of project.
- Your system must be controllable from the PC (typically over the JTAG)

Project Overview

- Everybody must do a different project: different microphone array, different number of microphones, different waveform, etc.
- You can propose a project: preferably in line with your current MSc research
- You need to design the DSP chain and choose appropriate system parameters: ADC sampling-rate, decimation (if any), architecture etc.
- Typically, you would build a sonar with angle-extraction, but you could also choose a communication or other type of project.
- Your system must be controllable from the PC (typically over the JTAG)

Project Overview

- Everybody must do a different project: different microphone array, different number of microphones, different waveform, etc.
- You can propose a project: preferably in line with your current MSc research
- You need to design the DSP chain and choose appropriate system parameters: ADC sampling-rate, decimation (if any), architecture etc.
- Typically, you would build a sonar with angle-extraction, but you could also choose a communication or other type of project.
- Your system must be controllable from the PC (typically over the JTAG)

The Report

Your report needs to:

- Describe the design overview
  (use a block diagram with some accompanying text)
- Provide minimal detail of the various modules that compose the system
- Provide practical results: performance figures, etc.
- Show that you have a working FPGA-based DSP chain
  (use photos, oscilloscope screenshots, etc. as evidence)
- Provide a link to your source code on an accessible Git server (typically GitHub)

The Report

Your report needs to:

- Describe the design overview
  (use a block diagram with some accompanying text)
- Provide minimal detail of the various modules that compose the system
- Provide practical results: performance figures, etc.
- Show that you have a working FPGA-based DSP chain
  (use photos, oscilloscope screenshots, etc. as evidence)
- Provide a link to your source code on an accessible Git server (typically GitHub)

The Report

Your report needs to:

- Describe the design overview
  (use a block diagram with some accompanying text)
- Provide minimal detail of the various modules that compose the system
- Provide practical results: performance figures, etc.
- Show that you have a working FPGA-based DSP chain
  (use photos, oscilloscope screenshots, etc. as evidence)
- Provide a link to your source code on an accessible Git server (typically GitHub)

The Report

Your report needs to:

- Describe the design overview
  (use a block diagram with some accompanying text)
- Provide minimal detail of the various modules that compose the system
- Provide practical results: performance figures, etc.
- Show that you have a working FPGA-based DSP chain
  (use photos, oscilloscope screenshots, etc. as evidence)
- Provide a link to your source code on an accessible Git server (typically GitHub)

The Report

Your report needs to:

- Describe the design overview
  (use a block diagram with some accompanying text)
- Provide minimal detail of the various modules that compose the system
- Provide practical results: performance figures, etc.
- Show that you have a working FPGA-based DSP chain
  (use photos, oscilloscope screenshots, etc. as evidence)
- Provide a link to your source code on an accessible Git server (typically GitHub)

Project Notes

- Remember that you are not designing a product
- You are designing a prototype,
  on a very small FPGA with limited resources
- Keep things simple – choose system parameters that favour easy implementation, not good system performance
- At the same time, the project must show FPGA competence,
  so don't make it too trivial
- Use the tools available, including the libraries and high-level design tools, where appropriate

Project Notes

- Remember that you are not designing a product
- You are designing a prototype,
  on a very small FPGA with limited resources
- Keep things simple – choose system parameters that favour easy implementation, not good system performance
- At the same time, the project must show FPGA competence,
  so don't make it too trivial
- Use the tools available, including the libraries and high-level design tools, where appropriate

Project Notes

- Remember that you are not designing a product
- You are designing a prototype,
  on a very small FPGA with limited resources
- Keep things simple – choose system parameters that favour easy implementation, not good system performance
- At the same time, the project must show FPGA competence,
  so don't make it too trivial
- Use the tools available, including the libraries and high-level design tools, where appropriate

Project Notes

- Remember that you are not designing a product
- You are designing a prototype,
  on a very small FPGA with limited resources
- Keep things simple – choose system parameters that favour easy implementation, not good system performance
- At the same time, the project must show FPGA competence,
  so don't make it too trivial
- Use the tools available, including the libraries and high-level design tools, where appropriate

Project Notes

- Remember that you are not designing a product
- You are designing a prototype,
  on a very small FPGA with limited resources
- Keep things simple – choose system parameters that favour easy implementation, not good system performance
- At the same time, the project must show FPGA competence,
  so don't make it too trivial
- Use the tools available, including the libraries and high-level design tools, where appropriate

Asking for Help

- I'm not on campus, but you can reach me via email
- Alternatively, open an issue on your GitHub fork and at-mention me (jpt13653903)
- Clearly explain what you're trying to do, and what you're struggling with
- Make sure that I can access your source code (typically by means of your Git repo)
- Where applicable, also include some pictures of the architecture you're trying to implement and a test-bench to highlight the problem
- You'll find some good resources on Google, but be very careful – more often than not the people don't know what they're talking about and lead you down the wrong path (ditto LLMs)

Asking for Help

- I'm not on campus, but you can reach me via email
- Alternatively, open an issue on your GitHub fork and at-mention me (jpt13653903)
- Clearly explain what you're trying to do, and what you're struggling with
- Make sure that I can access your source code (typically by means of your Git repo)
- Where applicable, also include some pictures of the architecture you're trying to implement and a test-bench to highlight the problem
- You'll find some good resources on Google, but be very careful – more often than not the people don't know what they're talking about and lead you down the wrong path (ditto LLMs)

Asking for Help

- I'm not on campus, but you can reach me via email
- Alternatively, open an issue on your GitHub fork and at-mention me (jpt13653903)
- Clearly explain what you're trying to do, and what you're struggling with
- Make sure that I can access your source code (typically by means of your Git repo)
- Where applicable, also include some pictures of the architecture you're trying to implement and a test-bench to highlight the problem
- You'll find some good resources on Google, but be very careful – more often than not the people don't know what they're talking about and lead you down the wrong path (ditto LLMs)

Asking for Help

- I'm not on campus, but you can reach me via email
- Alternatively, open an issue on your GitHub fork and at-mention me (jpt13653903)
- Clearly explain what you're trying to do, and what you're struggling with
- Make sure that I can access your source code (typically by means of your Git repo)
- Where applicable, also include some pictures of the architecture you're trying to implement and a test-bench to highlight the problem
- You'll find some good resources on Google, but be very careful – more often than not the people don't know what they're talking about and lead you down the wrong path (ditto LLMs)

Asking for Help

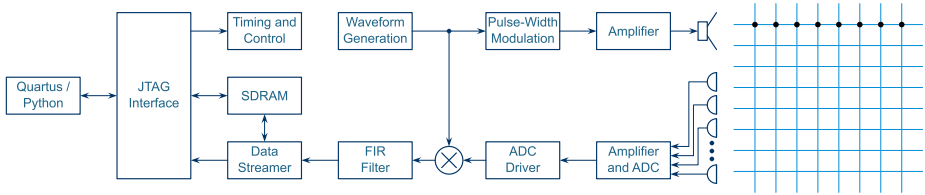
- I'm not on campus, but you can reach me via email
- Alternatively, open an issue on your GitHub fork and at-mention me (jpt13653903)
- Clearly explain what you're trying to do, and what you're struggling with
- Make sure that I can access your source code (typically by means of your Git repo)
- Where applicable, also include some pictures of the architecture you're trying to implement and a test-bench to highlight the problem
- You'll find some good resources on Google, but be very careful – more often than not the people don't know what they're talking about and lead you down the wrong path (ditto LLMs)

Asking for Help

- I'm not on campus, but you can reach me via email
- Alternatively, open an issue on your GitHub fork and at-mention me (jpt13653903)
- Clearly explain what you're trying to do, and what you're struggling with
- Make sure that I can access your source code (typically by means of your Git repo)
- Where applicable, also include some pictures of the architecture you're trying to implement and a test-bench to highlight the problem
- You'll find some good resources on Google, but be very careful – more often than not the people don't know what they're talking about and lead you down the wrong path (ditto LLMs)

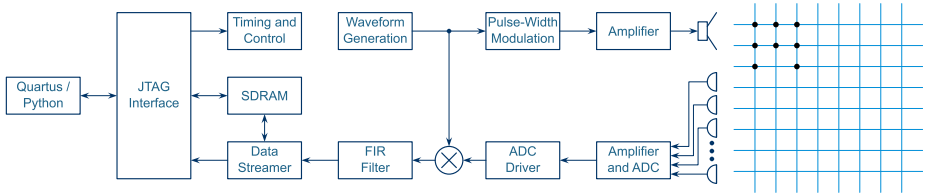
Project Ideas

Project 1 – FMCW Sonar with Full 2D Array



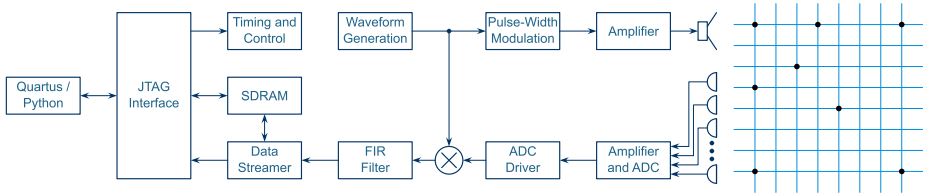
- Place the microphones as depicted above
- Receive, downconvert and stream the data to a PC
- Perform FFT-based azimuth image synthesis

Project 2 – FMCW Sonar with Small 3D Array



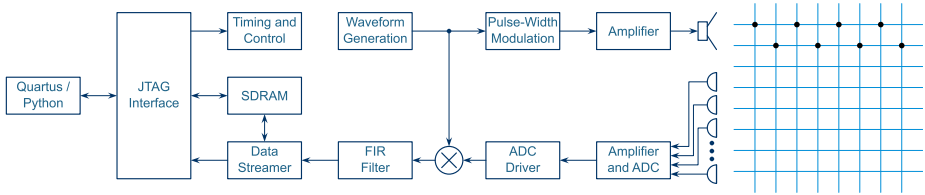
- Place the microphones as depicted above
- Receive, downconvert and stream the data to a PC
- Perform angle extraction on detected targets

Project 3 – FMCW Sonar with Sparse Array



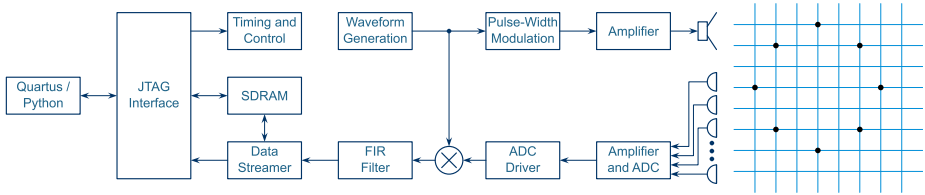
- Place the microphones as depicted above
- Receive, downconvert and stream the data to a PC
- Perform angle extraction on detected targets

Project 4 – FMCW Sonar with Triangular Array



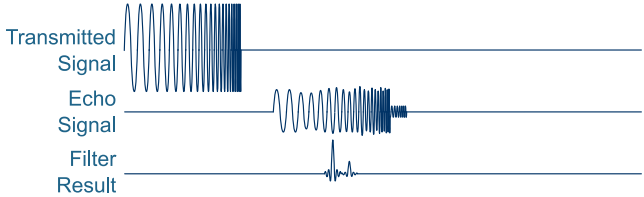
- Place the microphones as depicted above
- Receive, downconvert and stream the data to a PC
- Perform angle extraction on detected targets

Project 5 – FMCW Sonar with Circular Array



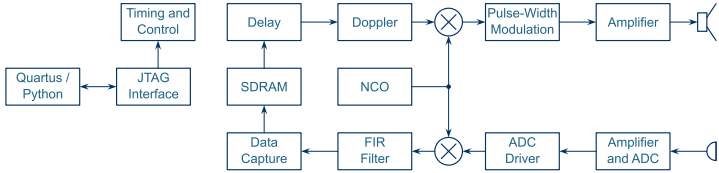
- Place the microphones as depicted above
- Receive, downconvert and stream the data to a PC
- Perform angle extraction on detected targets

Project 6 – Pulsed Sonar



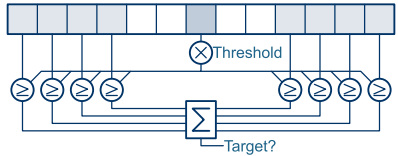
- Choose an appropriate array configuration
- Choose an appropriate pulsed waveform (typically a non-linear FM pulse)
- Receive, pulse-compress and stream the resulting data to a PC
- Perform angle extraction on detected targets

Project 7 – Sonar Beacon



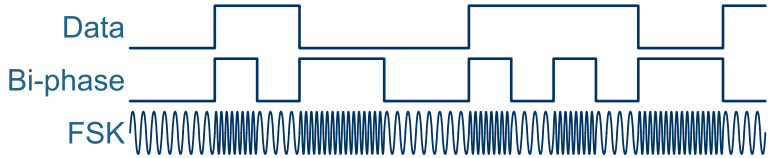
- Use a single microphone to receive signals from another sonar
- Implement a digital delay and Doppler shift
- Transmit the result through the speaker

Project 8 – Range CFAR



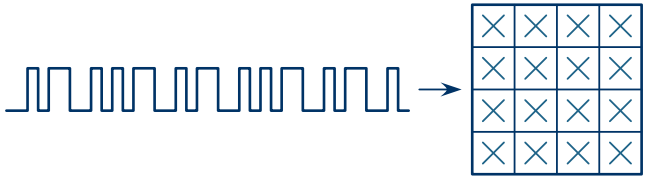
- Simulate FMCW data after the range-FFT (or get data from a friend doing one of the previous projects)
- Inject the data by means of the SDRAM
- Implement a range-CFAR and process the data
- Create a stream of detected targets
- Stream the resulting targets to the PC

Project 9 – FSK Communication



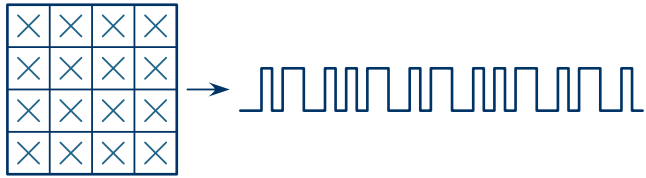
- This is a team project – one member must implement the transmit chain, and the other the receive chain
- Implement an FSK-based, bi-phase-coded communication channel
- Use S/PDIF as inspiration, with synchronisation word, etc.
- Show that you can transmit data (slowly) from one PC-DE10-SonarPCB combo to another

Project 10 – QAM Modulator



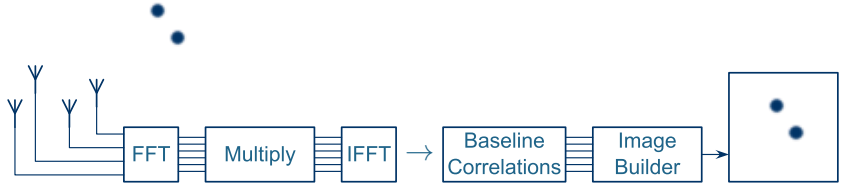
- Design and implement a 16-QAM modulator
- Inject a data stream and produce a 16-QAM output stream
- Include a synchronisation header and run-length limit
- Transmit the result over the speaker

Project 11 – QAM Demodulator



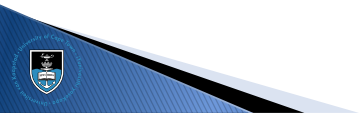
- Receive data from your colleague implementing Project 10
- Demodulate the data stream to obtain the original data

Project 12 – Astronomy Receiver



- Place multiple speakers around a room (multiple cell-phones, for example)
- Play pseudorandom signals (each one different) through the speakers
- Receive those signals using an appropriate microphone array
- Downconvert and send to the PC
- Perform correlation between all combinations of input channels
- Perform image synthesis

Outline



Design Once, Use Many

- Whenever possible, design your modules such that they can be re-used in other projects
- Use module parametrisation when appropriate
- Use standardised bus structures and interfaces, and the same interface family across all projects
- Use consistent naming conventions
- Clearly mark negative logic in the name

Libraries

- When possible, use the vendor-provided libraries
- Be careful: if the library function does not do quite what you want, it's generally easier and faster to design your own than to hack the existing library to do what you want
- Always use the correct tool for the job

Libraries

- When possible, use the vendor-provided libraries
- Be careful: if the library function does not do quite what you want, it's generally easier and faster to design your own than to hack the existing library to do what you want
- Always use the correct tool for the job

Testing and Debugging

- Always test as small a design as possible: in the ideal case, unit-test one module at a time
- In some cases, it's faster to simulate
- Other times, its easier and faster to compile and test on real hardware than to write the test-bench
- The on-chip logic analyser (i.e. Quartus Signal-tap, Vivado Chip-scope, Diamond Reveal, etc.) is your friend!

Scripting

- Most FPGA IDEs, including Lattice Diamond, Xilinx (AMD) Vivado and Altera (Intel) Quartus have powerful TCL scripting support
- You can automate version control, a more complicated compile chain (e.g. compiling a software component before the FPGA hardware), etc.
- Some IDEs lets you call a script automatically when starting and / or finishing a compile.

Scripting

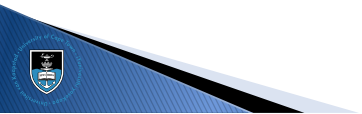
- Most vendor tool-chains have powerful TCL scripting support
- You can make scripts run automatically during the compilation process:

```
# In the QSF...

set_global_assignment         \
  -name PRE_FLOW_SCRIPT_FILE \
  "quartus_sh:Version Control/FirmwareVersion.tcl"

set_global_assignment          \
  -name POST_FLOW_SCRIPT_FILE \
  "quartus_sh:output_files/ProgramFPGA.tcl"
```

Outline

Course Conclusion

- We have covered a huge amount of work in a very short time
- It's up to you to go home and go play with the board
- And if you have questions: ask



Course Conclusion

- We have covered a huge amount of work in a very short time
- It's up to you to go home and go play with the board
- And if you have questions: ask

Course Conclusion

- We have covered a huge amount of work in a very short time
- It's up to you to go home and go play with the board
- And if you have questions: ask

Select References


Stephen Brown and Zvonko Vranesic
Fundamentals of Digital Logic with Verilog Design, 2nd Edition
ISBN 978-0-07-721164-6


Merrill L Skolnik
Introduction to RADAR Systems
ISBN 978-0-07-288138-7


Mark A. Richards and James A. Scheer
Principles of Modern Radar: Basic Principles
ISBN 978-1-89-112152-4


Deepak Kumar Tala
World of ASIC
http://www.asic-world.com/


Jean P. Nicolle
FPGA 4 Fun
http://www.fpga4fun.com/

FPGA Development for Radar, Radio-Astronomy and Communications

THE RADAR MASTERS COURSE

Dept. Electrical Engineering, University of Cape Town
Private Bag, Rondebosch, 7701, South Africa
http://www.rrsg.uct.ac.za

Presented by Dr John-Philip Taylor

Convened by Dr Stephen Paine

Day 5 – 7 August 2026